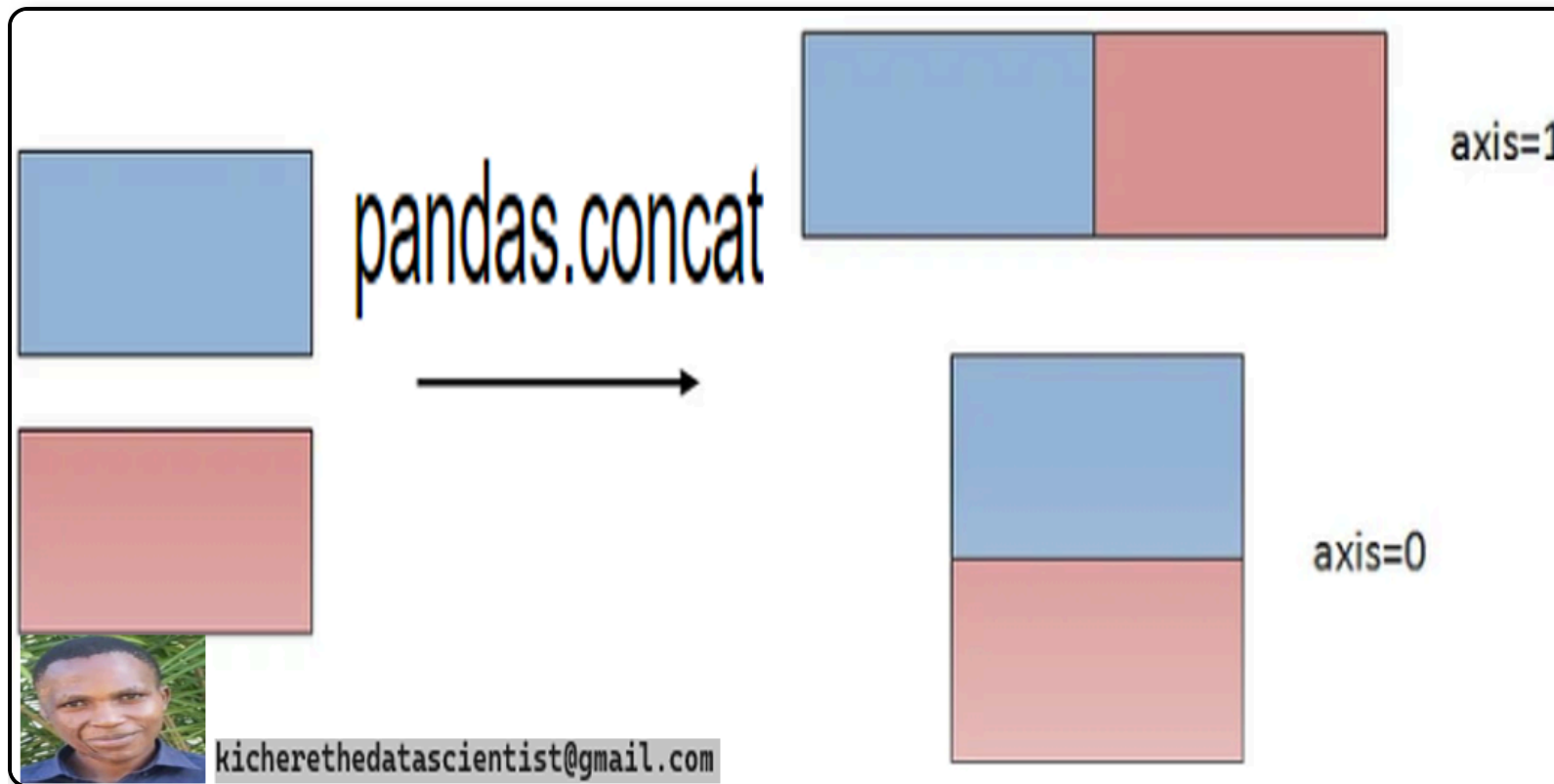


None



1st Edition

Why This E-book?

"The aim of this ebook is to give you the 'aha' moment right away at the start of learning a new concept"

- Practical step By Step Guide With Simple Examples
- Visual Illustrations and Interactive
- Simple Datasets
- Comprehensive Coverage(pandas Documentation used as reference)

Contents

The "objs" Parameter

The "axis" Parameter

The "join" Parameter

The "ignore_index" Parameter

The "keys" Parameter

The "levels" Parameter

The "names" Parameter

[The "verify_integrity" Parameter](#)

[The "sort" Parameter](#)

[The "copy" Parameter](#)

Preface

[What Makes This Ebook Unique](#)

[Author Biography](#)

[Other ebooks](#)

[Contacts and Social Media](#)

[Sources & References](#)

[Acknowledgments](#)

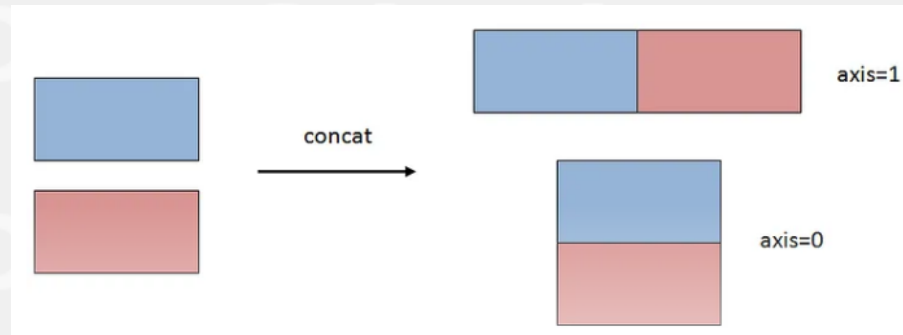
[Thank You!](#)

```
#Import Libraries
import numpy as np
import pandas as pd
```

Introduction

pandas.concat:

Concatenate pandas objects along a particular axis.



```
#some synonyms for "concatenate":
#Combine, Join, Link, Merge, Append, Connect, Fuse, Integrate, Attach, Chain
```

The "pandas.concat" Syntax

```
pandas.concat(objs, *, axis=0, join='outer', ignore_index=False, keys=None,
              levels=None, names=None, verify_integrity=False, sort=False, copy=None)
```

Parameters

The **objs** parameter

The **objs** parameter in the **pandas.concat()** function stands for "objects" and refers to a sequence (list or similar iterable) of pandas objects(e.g DataFrames) that you want to concatenate together. These objects could be DataFrames, Series, or even panels (though panels are less commonly used now).

The **axis** parameter

The "axis" parameter is an axis to concatenate along.

Concatenating Along Rows(Vertically) (Default axis=0)

```
# Define three Series
s1 = pd.Series([1, 2, 3], name='A', index=['a', 'b', 'c'])
s2 = pd.Series(['x', 'y', 'z'], name='B')
s3 = pd.Series([7, 8, 9], name='C', index=['p', 'q', 'r'])
```

s1

a 1
b 2
c 3

s2

0 x
1 y

Output:

a 1
b 2
c 3
0 x
1 y

Code Snippet:

`pd.concat([s1, s2, s3])`

or

`pd.concat([s1, s2, s3], axis= 0)`

```

2    z
s3
p    7
q    8
r    9
2    z
p    7
q    8
r    9
or
pd.concat([s1, s2, s3], axis='index' )

```

Concatenating Along Columns(side by side) (axis=1)

```

# Define two dataframes
df1 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
df2 = pd.DataFrame({'C': ['u', 'v', 'w'], 'D': ['x', 'y', 'z']})

```

df1:			df2:			Output:				
	A	B		C	D		A	B	C	D
0	1	4	0	u	x	0	1	4	u	x
1	2	5	1	v	y	1	2	5	v	y
2	3	6	2	w	z	2	3	6	w	z

→

Code Snippet:

```

pd.concat([df1, df2], axis=1)
or
pd.concat([df1, df2], axis='columns')

```

Concatenating a Mapping of Series

```

#Understanding Mapping in pandas
#In pandas, a mapping typically refers to a Python dictionary where keys are used to access corresponding values.
#When working with pandas functions like concat(), which combines multiple DataFrame objects, you can pass a
#mapping (dictionary) as an argument. Here's how it works:

```

```
# Create three Series
s1 = pd.Series([1, 2, 3], name='A')
s2 = pd.Series([4, 5, 6], name='B')
s3 = pd.Series([7, 8, 9], name='C')
display(s1, s2, s3)
```

```
0    1
1    2
2    3
Name: A, dtype: int64
0    4
1    5
2    6
Name: B, dtype: int64
0    7
1    8
2    9
Name: C, dtype: int64
```

```
# Create a dictionary mapping
series_dict = {'col1': s1, 'col2': s2, 'col3': s3}
print(series_dict)
```

```
{'col1': 0    1
1    2
2    3
Name: A, dtype: int64, 'col2': 0    4
1    5
2    6
Name: B, dtype: int64, 'col3': 0    7
1    8
2    9
Name: C, dtype: int64}
```

```
##If a mapping is passed, the sorted keys will be used as the
##keys argument, unless it is passed, in which case the values will be selected
```

```
# Concatenate the Series along columns (axis=1)
pd.concat(series_dict, axis=1)
```

	col1	col2	col3
0	1	4	7

	col1	col2	col3
1	2	5	8
2	3	6	9

*##Can also add a layer of hierarchical indexing on the concatenation axis,
##which may be useful if the labels are the same (or overlapping)
##on the passed axis number.*

```
# Create three DataFrames
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
df3 = pd.DataFrame({'A': [9, 10], 'B': [11, 12]})
display(df1, df2, df3)
```

	A	B
0	1	3
1	2	4

	A	B
0	5	7
1	6	8

	A	B
0	9	11
1	10	12

```
# Create a dictionary mapping
df_dict = {'DF1': df1, 'DF2': df2, 'DF3': df3}

# Concatenate the DataFrames along rows (axis=0)
pd.concat(df_dict, axis=0)
```

		A	B
DF1	0	1	3
	1	2	4
DF2	0	5	7
	1	6	8
DF3	0	9	11
	1	10	12

#In summary, when you pass a dictionary (mapping) to functions like concat() in pandas:

#The keys of the dictionary are used as identifiers or labels.

#They can form a hierarchical index in the resulting DataFrame.

#Pandas sorts these keys by default if you don't specify a different order.

##Any None objects will be dropped silently unless they are

##all None in which case a ValueError will be raised.

Creating some sample DataFrames

```
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
```

```
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
```

```
(df1, df2)
```

```
(  A  B
```

```
0  1  3
```

```
1  2  4,
```

```
   A  B
```

```
0  5  7
```

```
1  6  8)
```

Creating a List with None in it

```
dfs = [df1, None, df2, None]
```

```
dfs
```

```
[  A  B
```

```
0  1  3
```

```
1  2  4,
```

```
None,
```

```
   A  B
```

```
0 5 7
1 6 8,
None]
```

```
# Concatenating the DataFrames using pd.concat
pd.concat(dfs)
```

	A	B
0	1	3
1	2	4
0	5	7
1	6	8

```
##all None in which case a ValueError will be raised.
```

```
#Example: ValueError When All Objects are None
#In this example, we'll explicitly create a situation where all objects
#passed to pd.concat are None, which will result in a ValueError.

# Creating a list with all None objects
dfs_all_none = [None, None, None]
#pd.concat(dfs_all_none)
```

The **join** parameter

**The "join" parameter handle indexes on other axis (or axes).
{'inner', 'outer'}, default 'outer'.**

```
#'inner': This performs an inner join operation along the specified axis.
#It returns only the intersection of the indexes (or keys) from the
#objects being concatenated. Rows (or columns) with mismatched
#keys are excluded from the result.
```

Concatenate DataFrames along axis=1 with "inner" join

Example DataFrames

```
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]}, index=['X', 'Y'])
df2 = pd.DataFrame({'C': ['a', 'b'], 'D': ['c', 'd']}, index=['Y', 'Z'])
```

df1:

	A	B
X	1	3
Y	2	4

df2:

	C	D
Y	a	c
Z	b	d

Output:

	A	B	C	D
Y	2	4	a	c

Code Snippet:

```
pd.concat([df1, df2], axis=1, join='inner')
```

Concatenate DataFrames along axis=0 with "inner" join

df1:

	A	B
X	1	3
Y	2	4

df2:

	B	D
Y	a	c
Z	b	d

Output:

	B
X	3
Y	4
Y	a
Z	b

Code Snippet:

```
pd.concat([df1, df2], axis=0, join='inner')
```

*#It returns the union of the indexes (or keys) from the objects being concatenated.
 #It includes rows (or columns) from all objects, filling missing values with
 #NaN (or a specified fill value) where there are no matching keys*

Concatenate DataFrames along axis=1 with "outer" join

df1: df2: Output: Code Snippet:

	A	B
X	1	3
Y	2	4

	C	D
Y	a	c
Z	b	d

→

	A	B	C	D
X	1.0	3.0	NaN	NaN
Y	2.0	4.0	a	c
Z	NaN	NaN	b	d

```
pd.concat([df1, df2], axis=1, join='outer')
or
pd.concat([df1, df2], axis=1)
```

Concatenate DataFrames along axis=0 with "outer" join

df1: df2: Output: Code Snippet:

	A	B
X	1	3
Y	2	4

	B	D
Y	a	c
Z	b	d

→

	A	B	D
X	1.0	3	NaN
Y	2.0	4	NaN
Y	NaN	a	c
Z	NaN	b	d

```
pd.concat([df1, df2], axis=0, join='outer')
or
pd.concat([df1, df2], axis=0)
```

The `ignore_index` parameter

"ignore_index" is a boolean and default False

```
df1 = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})
df2 = pd.DataFrame({'A': ['x', 'y', 'z'], 'B': ['p', 'q', 'r']})
```

#Setting ignore_index=True:

*#When ignore_index=True is specified, the resulting object (DataFrame or Series)
#will ignore the original indices. Instead, it generates a new integer index for
#the concatenated or appended object. This new index starts from 0 and increments
#sequentially, ensuring each row or item has a unique index*

If True, do not use the index values along the concatenation axis.

The resulting axis will be labeled 0, ..., n - 1.

This is useful if you are concatenating objects where the concatenation axis does not have meaningful indexing information.

Note the index values on the other axes are still respected in the join.

DataFrames:

	A	B
0	1	4
1	2	5
2	3	6

	A	B
0	x	p
1	y	q
2	z	r



Output:

	A	B
0	1	4
1	2	5
2	3	6
3	x	p
4	y	q
5	z	r

Code Snippet:

```
pd.concat([df1, df2], ignore_index = True)
```

DataFrames:

ignore_index=False (default behavior):

This preserves the original indices.

Rows or items from the concatenated or appended objects maintain their original indices, leading to potentially duplicated or overlapping indices in the resulting object.

	A	B
0	1	4
1	2	5
2	3	6

	A	B
0	x	p
1	y	q
2	z	r



Output:

	A	B
0	1	4
1	2	5
2	3	6
0	x	p
1	y	q
2	z	r

Code Snippet:

```
d.concat([df1, df2], ignore_index = False)
or
pd.concat([df1, df2])
```

The keys parameter

The "keys" parameter accepts a sequence and it is None by default.

"For multiple levels, use tuples to define them. The first set of keys will be used as the highest level in the hierarchy."

Single Level Concatenation

```
# Define two dataframes
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': ['a', 'b'], 'B': ['c', 'd']})
```

In Single Level Concatenation: we create a hierarchical index where df1 and df2 are the outermost level labels.

df1:

	A	B
0	1	3

Output:

Code Snippet:

```
pd.concat([df1, df2], keys=['df1', 'df2'])
```

	A	B
1	2	4

	A	B
0	a	c
1	b	d

	df1	A	B
0	1	2	4

	df2	A	B
0	a	c	
1	b	d	

Multi-Level Concatenation

Here, ('numbers', 'df1') and ('letters', 'df2') create a multi-level column index with two levels: 'numbers' and 'letters', with 'df1' and 'df2' as the second level..

Output:

Code Snippet:

```
pd.concat([df1, df2], axis=1, keys=[('numbers', 'df1'), ('letters', 'df2')])
```

	A	B
0	1	3
1	2	4

	C	D
0	a	c
1	b	d

	numbers		letters	
	df1		df2	
	A	B	C	D
0	1	3	a	c
1	2	4	b	d

```
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': ['a', 'b'], 'B': ['c', 'd']})
```

df1:

Output:

	A	B
0	1	3
1	2	4



	A	B
0	a	c
1	b	d

numbers	df1		A	B
		0	1	3
		1	2	4
letters	df2		A	B
		0	a	c
		1	b	d

Code Snippet:

```
pd.concat([df1, df2], axis=0, keys=[('numbers', 'df1'), ('letters', 'df2')])
```

The levels parameter

The "levels" parameter accepts list of sequences and it is None by default.

Specific levels (unique values) to use for constructing a MultiIndex. Otherwise they will be inferred from the keys."

```
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': ['a', 'b'], 'B': ['c', 'd']})
```

DataFrames:

	A	B
0	1	3
1	2	4

	A	B
0	a	c
1	b	d



Output:

		A	B
first	0	1	3
	1	2	4
second	0	a	c
	1	b	d

Code Snippet:

```
pd.concat(
    [df1, df2],
    keys=['first', 'second'],
    levels=[['first', 'second']]
)
```

The `names` parameter

The "names" parameter accepts a list, and it is None by default. It defines the names for the levels in the resulting hierarchical index.

DataFrames:

	A	B
0	1	3
1	2	4

	A	B
0	a	c
1	b	d



Output:

	dfs	idxs	A	B
df1	0	1	3	
	1	2	4	
df2	0	a	c	
	1	b	d	

Code Snippet:

```
pd.concat([df1, df2], keys=['df1', 'df2'], names=['dfs', 'idxs'])
```

The verify_integrity parameter

The "verify_integrity" parameter is boolean, and defaults to False. It checks whether the new concatenated axis contains duplicates. This can be very expensive relative to the actual data concatenation.

```
# Define two dataframes
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]}, index=['a', 'b'])
df2 = pd.DataFrame({'A': ['a', 'b'], 'B': ['c', 'd']}, index=['b', 'c'])
```

When verify_integrity=False (default):

Concatenation proceeds without checking for duplicate indices. you can see that index 'b' appears twice. pd.concat does not raise any errors or warnings by default, but there are duplicate indices.

DataFrames:

Output:

A	B
a	1 3
b	2 4

→

A	B
a	1 3
b	2 4
b	a c
c	b d

A	B
b	a c
c	b d

Code Snippet:

```
pd.concat([df1, df2], verify_integrity=False)
```

When `verify_integrity=True`:

Pandas checks for duplicate indices after concatenation. If duplicates are found, it raises a `ValueError`, ensuring that the final `DataFrame` does not have duplicate indices.

The `sort` parameter

**The "sort" parameter is boolean, and defaults to False.
It is used to sort non-concatenation axis if it is not already aligned.**

One exception to this is when the non-concatenation axis is a `DatetimeIndex` and `join='outer'` and the axis is not already aligned. In that case, the non-concatenation axis is always sorted lexicographically.

```
# Create two DataFrames with different index orders
df1 = pd.DataFrame({'B': [1, 2, 3], 'A': [4, 5, 6]}, index=['a', 'b', 'c'])
df2 = pd.DataFrame({'A': [7, 8], 'B': [9, 10]}, index=['c', 'b'])
```

Here, non-concatenation axis is 1 or 'columns' it is aligned to 'A', 'B'.

DataFrames:

"sort=False": "sort=True":

B	A		B	A		A	B	
a	1	4	a	1	4	a	4	1
b	2	5	b	2	5	b	5	2
c	3	6	c	3	6	c	6	3
			c	9	7	c	7	9
			b	10	8	b	8	10
A	B							
c	7	9						
b	8	10						

Code Snippets:

```
pd.concat([df1, df2], axis=0, sort=False)
and
pd.concat([df1, df2], axis=0, sort=True)
```

Exception:

"Here, non-concatenation axis is 0 and the outputs for 'True' or 'False' are the same meaning that when the non-concatenation axis is a DatetimeIndex and join='outer' and the axis is not already aligned. In that case, the non-concatenation axis is always sorted lexicographically."

```
# Define two dataframes
df1 = pd.DataFrame({
    'A': [10, 20, 30],
    index=pd.to_datetime(['2024-08-01', '2024-08-02', '2024-08-03'])
})
df2 = pd.DataFrame({
    'B': [70, 80, 90],
    index=pd.to_datetime(['2024-07-01', '2024-08-01', '2024-08-04'])
})
```

df1:		df2:			"sort=False":		"sort=True":			
A		B			A	B	A	B		
					2024-07-01	NaN	70.0	2024-07-01	NaN	70.0
2024-08-01	10	2024-07-01	70	→	2024-08-01	10.0	80.0	2024-08-01	10.0	80.0
2024-08-02	20	2024-08-01	80		2024-08-02	20.0	NaN	2024-08-02	20.0	NaN
2024-08-03	30	2024-08-04	90		2024-08-03	30.0	NaN	2024-08-03	30.0	NaN

Code Snippets:

```
pd.concat([df1, df2], axis=1,
          join="outer", sort=False)
and
```

		A	B			A	B
2024-08-04		NaN	90.0	2024-08-04		NaN	90.0

```
pd.concat([df1, df2], axis=1, join="outer", sort=True)
```

@Kichere The Data Scientist

The `copy` parameter

The "copy" parameter is boolean, and defaults to True.

- when `copy=True`: pandas creates a new copy of the data, ensuring that the original DataFrames remain unchanged.
- when `copy=False`: pandas avoids unnecessary copies, which can be more efficient but may have implications if you need to ensure the original DataFrames are not altered. In practice, `copy=False` can be beneficial for large datasets where performance and memory usage are critical, while `copy=True` ensures safer, isolated operations.

Sources & References

[pandas.concat Documentation](https://pandas.pydata.org/docs/reference/api/pandas.concat.html) (https://pandas.pydata.org/docs/reference/api/pandas.concat.html)

Acknowledgments

First and foremost, I express my gratitude to **God** for His blessings and guidance throughout this journey. I am also deeply thankful to the **Eastern Africa Statistical Training Centre** and **SKT Tanzania Ltd** for their invaluable support. Special thanks to my **parents** whose unwavering encouragement and support have been the cornerstone of this endeavor.

Other E-books

[Download](https://github.com/kicherethedatascientist/My_e-Books) (https://github.com/kicherethedatascientist/My_e-Books)

Author Biography

Kichere Magubu is a data enthusiast and content creator.

Contacts and Social Media

Kichere Magubu

 Dar es salaam, Tanzania

 [Kichere Magubu](#)

 [Kichere The Data Scientist](#)

 [KichereTheDataScientist](#)

 [KichereTheDataScientist](#)

✉ kicherethedatascientist@gmail.com

📷 kicherethedatascientist

☎ +255 654 729 851

Powered by

Eastern Africa Statistical Training Centre

SKT Tanzania Ltd

Thank you!

Thank you for reading this e-book!.

If you found this book valuable and beneficial, I kindly request that you take a moment to review it. Your honest feedback is highly appreciated and makes a significant difference to me.

Please take a minute to write me an honest review. Your support means the world to me!

Thank you!

```
#print("The cell to convert jupyter notebook to html")
#!jupyter nbconvert --to hide_code_html "Pivot Tables.ipynb"
```